

## ★ List Interface :

- List is a subinterface of collection in java.
- Duplicates are allowed with index (0-based)

### Syntax:-

List < DataType > listName = new ArrayList <> ();

↓  
if we give class name here, then we can give <sup>use</sup> any datatype inside this list.

## ★ ArrayList :

- A dynamic array that grows automatically.
- index-based access.

## ★ LinkedList :

- it is a doubly linked list. (each list connected with another list)
- Best for insertion and deletion.

## ★ Vector :

- It is a dynamic array.
- It is synchronised and thread-safe.
- Doubles its size when needed.

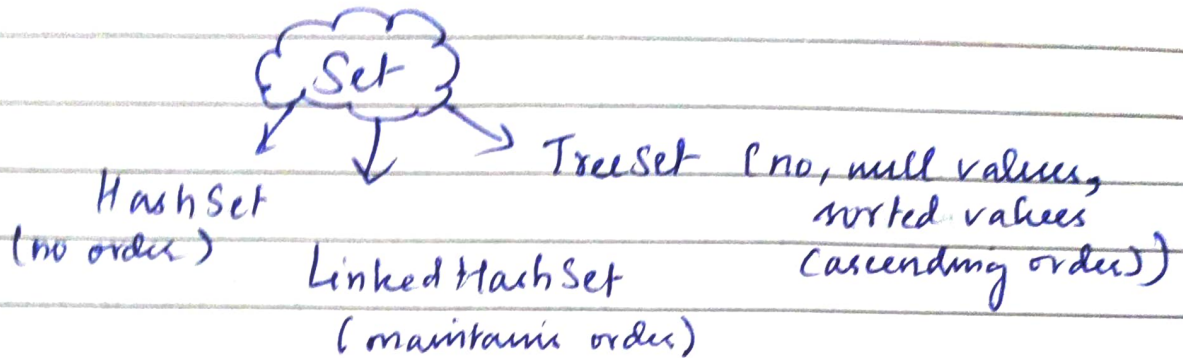
## ★ Stack :

- Follows LIFO (Last in first out) order for storing & retrieving elements.
- It inherits from vector.

## \* Set :

- consist of unique elements.
- Doesn't allow duplicates & doesn't maintain insertion order.
- at most one null element.

`Set < Type > set = new HashSet <> ();`



## \* ~~Queue~~ <sup>Map</sup> Interface :

- It stores values in key-value pairs.

Syntax :-

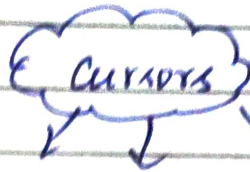
`Map < Key Type, Value Type > map = new HashMap<>();`

- No duplicates for keys but values can be duplicated.

## \* Queue Interface :

- FIFO (First in First Out) → it is designed to hold elements before processing.

## \* Cursor :



Iterators

↓  
It moves forward only.

(List, Set, Queue)

- used to traverse elements, allows remove().

• hasNext()

next()

remove()

• [A] → [B] → [C] → [D]



ListIterator



- used for List (ArrayList, LinkedList) and supports two way.

• add()

remove()

set()

- More powerful than iterator.

- Moves in both directions (forward & backward).

• [A] ⇌ [B] ⇌ [C] ⇌ [D]



Enumeration

- legacy cursor used only for old classes.

- Vector, Hashtable, Stack.

- Moves forward only, can't remove elements.

- hasMoreElements()  
nextElement();